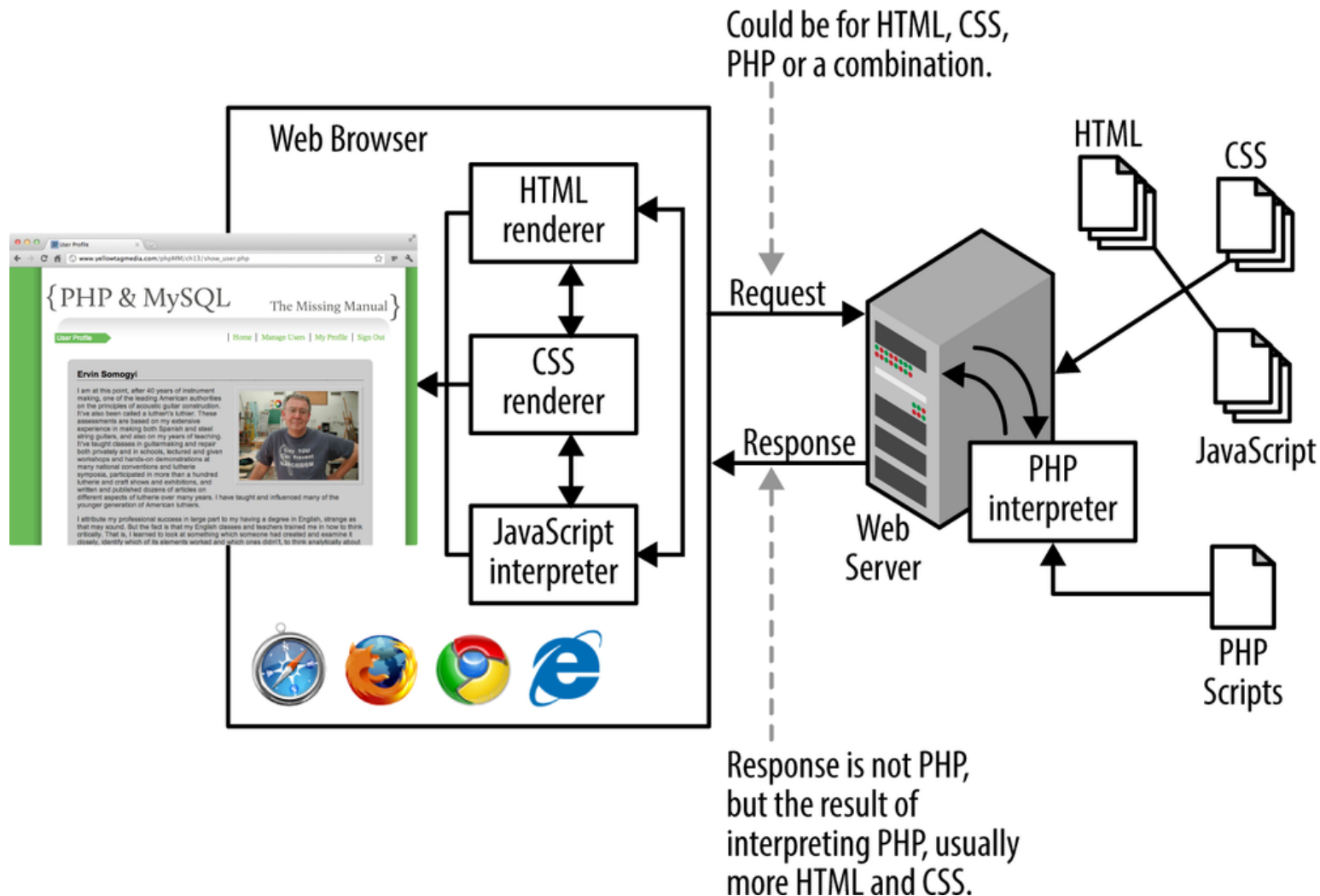




PHP INTRODUCTION 2025

Silvio Priem Mendes, PhD

WAMP/HTTP Architecture



- **DataBase Server not represented at this point**

PHP Introduction

- PHP is a server side scripting language that is embedded in HTML. It is used to manage dynamic content, databases, session tracking, even build entire webapps.
- It is integrated with a number of popular databases, including MySQL, PostgreSQL, Oracle, and Microsoft SQL Server.
- PHP is fast in its execution, especially when compiled as an Apache module.
- PHP supports a large number of major protocols such as POP₃, IMAP, and LDAP.
- PHP is forgiving: PHP language tries to be as forgiving as possible.
- PHP Syntax is C-Like.

PHP Common uses

- PHP performs system functions, i.e. from files on a system it can create, open, read, write, and close them.
- PHP can handle forms, i.e. gather data from files, save data to a file, through email you can send data, return data to the user.
- You add, delete, modify elements within your database through PHP.
- Access cookies variables and set cookies.
- Using PHP, you can restrict users to access some pages of your website.
- It can encrypt data.

PHP Scripts

- Typically file ends in **.php** (web server configuration);
- Separated in files with the **<?php ... ?>** tag;
- PHP code can make up an entire file, or can be contained in html; this is a choice for now....more on later regarding MVC;
- A PHP block can be placed anywhere in the document;
- Program lines end in **;"** or you get an error;
- Server recognizes embedded script and executes the PHP;
- Result is passed to browser, PHP source isn't visible.

|| 'Hello World' Script in PHP

- Example includes comments

```
<!DOCTYPE html>
<html>
  <body>
    <h1>My first PHP page</h1>
    <?php
      echo 'Hello World!';
      // This is a single-line comment
      # This is also a single-line comment
      /*
        This is a multiple-lines comment
        block that spans over multiple
        lines
      */
    ?>
  </body>
</html>
```

"Hello World" Script in PHP

PHP Source

```
<html>

<head>
<title>Hello World</title>
</head>
<body>

<?php
$name = "World";
echo "<h1>Hello, $name!</h1>";
?>

</body>
</html>
```

View in
browser

Result in browser

Hello, World!

View
source

Browser Source

```
<html>

<head>
<title>Hello World</title>
</head>
<body>
<h1>Hello, World!</h1>
</body>
</html>
```

PHP Comments

- A comment in PHP code is a line that is not read/executed as part of the program.
 - Its only purpose is to be read by someone who is looking at the code.
- **Comments can be used to:**
 - Let others understand what you are doing
 - Remind yourself of what you did - Most programmers have experienced coming back to their own work a year or two later and having to re-figure out what they did. Comments can remind you of what you were thinking when you wrote the code

PHP Case Sensitivity

- In PHP, all keywords (e.g. if, else, while, echo, etc.) and built-in functions, **are NOT case-sensitive**.
- **However; all variable names are case-sensitive.**
 - In the example below, only the first statement will display the value of the \$color variable (this is because \$color, \$COLOR, and \$coLoR are treated as three different variables):

```
<!DOCTYPE html>
<html>
<body>
<?php
    $color = 'red';
    echo 'My car is ' . $color . '<br>';
    echo 'My house is ' . $COLOR . '<br>';
    echo 'My boat is ' . $coLoR . '<br>';
?>
</body>
</html>
```

Declaring PHP Variables

- In PHP, a variable starts with the \$ sign, followed by the name of the variable:

```
<?php
    $txt = 'Hello world!';
    $x = 5;
    $y = 10.5;
?>
```

- After the execution of the statements above, the variable **\$txt** will hold the value **Hello world!**, the variable **\$x** will hold the value **5**, and the variable **\$y** will hold the value **10.5**;
- **Note:** Unlike other programming languages, PHP has no command for declaring a variable. It is created the moment you first assign a value to it.

PHP Variables

- A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume) **RECOMMENDED**.

Rules for PHP variables:

- A variable starts with the \$ sign, followed by the name of the variable
- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (\$age and \$AGE are two different variables)

PHP Variables

- **PHP is a Loosely Typed Language**

- In the previous example, notice that we did not have to tell PHP which data type the variable is.
- PHP automatically converts the variable to the correct data type, depending on its value.

PHP Data Types

- Variables can store data of different types, and different data types can do different things.
- **PHP supports the following data types:**
 - String
 - Integer
 - Float (floating point numbers - also called double)
 - Boolean
 - Array
 - Object
 - NULL

PHP String

- A string is a sequence of characters, like "Hello world!".
- A string can be any text inside quotes. You can use single or double quotes (**although there are differences**):

```
<?php
    $x = "Hello world!";
    $y = 'Hello world!';

    echo $x;
    echo "<br>";
    echo $y;
?>
```

PHP String

- Single Quoted

```
$color = 'Green' ;
```

- Double Quoted (strings enclosed by " are interpreted)

```
// outputs Green Eggs and Ham  
$food = "$color Eggs and Ham";  
echo $food;
```

PHP Integer

- An integer data type is a non-decimal number between -2,147,483,648 and 2,147,483,647.

- **Rules for integers:**

- An integer must have at least one digit
- An integer must not have a decimal point
- An integer can be either positive or negative
- Integers can be specified in three formats: decimal (10-based), hexadecimal (16-based - prefixed with ox) or octal (8-based - prefixed with o)

PHP Integer

- In the following example \$x is an integer.
- The PHP **var_dump()** function returns the data type and value:

```
<?php
    $x = 5985;
    var_dump($x);
?>
```

PHP Float

- A float (floating point number) is a number with a decimal point or a number in exponential form.
- In the following example \$x is a float.
- The PHP **var_dump()** function returns the data type and value:

```
<?php
    $x = 10.365;
    var_dump($x);
?>
```

PHP Boolean

- A Boolean represents two possible states: TRUE or FALSE.

```
$x = true;  
$y = false;
```

- Booleans are often used in conditional testing.
- You will learn more about conditional testing in a later chapter of this course.

PHP Object

- An object is a data type which stores data and information on how to process that data.
 - **In PHP, an object must be explicitly declared.**
 - First we must declare a class of object. For this, we use the class keyword. A class is a structure that can contain properties and methods:

```
<?php
class Car {
    function Car() {
        $this->model = 'VW';
    }
}
// create an object
$herbie = new Car();
// show object properties
echo $herbie->model;
?>
```

- You will learn more about objects in a later chapter of this course.

PHP NULL

- Null is a special data type which can have only one value: **NULL**.
- A variable of data type NULL is a variable that has no value assigned to it.
 - **Note:** If a variable is created without a value, it is automatically assigned a value of NULL.
- Variables can also be emptied by setting the value to NULL:

```
<?php
    $x = 'Hello world!';
    $x = null;
    var_dump($x);
?>
```

PHP Operators

- Operators are used to perform operations on variables and values.
- **PHP divides the operators in the following groups:**
 - Arithmetic operators
 - Assignment operators
 - Comparison operators
 - Increment/Decrement operators
 - Logical operators
 - String operators
 - Array operators (explained later)

PHP Arithmetic Operators

- The PHP arithmetic operators are used with numeric values to perform common arithmetical operations, such as addition, subtraction, multiplication etc.

Operator	Name	Example	Result
+	Addition	$\$x + \y	Sum of $\$x$ and $\$y$
-	Subtraction	$\$x - \y	Difference of $\$x$ and $\$y$
*	Multiplication	$\$x * \y	Product of $\$x$ and $\$y$
/	Division	$\$x / \y	Quotient of $\$x$ and $\$y$
%	Modulus	$\$x \% \y	Remainder of $\$x$ divided by $\$y$
**	Exponentiation	$\$x ** \y	Result of raising $\$x$ to the $\$y$ 'th power (Introduced in PHP 5.6)

PHP Assignment Operators

- The PHP assignment operators are used with numeric values to write a value to a variable.
 - The basic assignment operator in PHP is "=". It means that the left operand gets set to the value of the assignment expression on the right.

Assignment	Same as...	Description
<code>x = y</code>	<code>x = y</code>	The left operand gets set to the value of the expression on the right
<code>x += y</code>	<code>x = x + y</code>	Addition
<code>x -= y</code>	<code>x = x - y</code>	Subtraction
<code>x *= y</code>	<code>x = x * y</code>	Multiplication
<code>x /= y</code>	<code>x = x / y</code>	Division
<code>x %= y</code>	<code>x = x % y</code>	Modulus

PHP Comparison Operators

- The PHP comparison operators are used to compare two values (number or string):

Operator	Name	Example	Result
==	Equal	<code>\$x == \$y</code>	Returns true if \$x is equal to \$y
===	Identical	<code>\$x === \$y</code>	Returns true if \$x is equal to \$y, and they are of the same type
!=	Not equal	<code>\$x != \$y</code>	Returns true if \$x is not equal to \$y
<>	Not equal	<code>\$x <> \$y</code>	Returns true if \$x is not equal to \$y
!==	Not identical	<code>\$x !== \$y</code>	Returns true if \$x is not equal to \$y, or they are not of the same type
>	Greater than	<code>\$x > \$y</code>	Returns true if \$x is greater than \$y
<	Less than	<code>\$x < \$y</code>	Returns true if \$x is less than \$y
>=	Greater than or equal to	<code>\$x >= \$y</code>	Returns true if \$x is greater than or equal to \$y
<=	Less than or equal to	<code>\$x <= \$y</code>	Returns true if \$x is less than or equal to \$y

PHP Inc/dec Operators

- The PHP increment operators are used to increment a variable's value.
- The PHP decrement operators are used to decrement a variable's value.

Operator	Name	Description
++\$x	Pre-increment	Increments \$x by one, then returns \$x
\$x++	Post-increment	Returns \$x, then increments \$x by one
--\$x	Pre-decrement	Decrements \$x by one, then returns \$x
\$x--	Post-decrement	Returns \$x, then decrements \$x by one

PHP Logical Operators

- The PHP logical operators are used to combine conditional statements.

Operator	Name	Example	Result
and	And	<code>\$x and \$y</code>	True if both <code>\$x</code> and <code>\$y</code> are true
or	Or	<code>\$x or \$y</code>	True if either <code>\$x</code> or <code>\$y</code> is true
xor	Xor	<code>\$x xor \$y</code>	True if either <code>\$x</code> or <code>\$y</code> is true, but not both
<code>&&</code>	And	<code>\$x && \$y</code>	True if both <code>\$x</code> and <code>\$y</code> are true
<code> </code>	Or	<code>\$x \$y</code>	True if either <code>\$x</code> or <code>\$y</code> is true
<code>!</code>	Not	<code>!\$x</code>	True if <code>\$x</code> is not true

PHP String Operators

- PHP has two operators that are specially designed for strings.

Operator	Name	Example	Result
.	Concatenation	<code>\$txt1 . \$txt2</code>	Concatenation of <code>\$txt1</code> and <code>\$txt2</code>
<code>.=</code>	Concatenation assignment	<code>\$txt1 .= \$txt2</code>	Appends <code>\$txt2</code> to <code>\$txt1</code>

PHP Type Coercion

- Just like javascript, php is loosely typed;
- Coercion occurs the same way;
 - If you concatenate a number and string, the number becomes a string;

```
<?php
    $foo = '1';    // $foo is string (ASCII 49)
    $foo *= 2;     // $foo is now an integer (2)
    $foo = $foo * 1.3; // $foo is now a float (2.6)
    $foo = 5 * '10 Little Piggies'; // $foo is integer (50)
    $foo = 5 * '10 Small Pigs';      // $foo is integer (50)
?>
```

PHP Constants

- Constants are like variables except that once they are defined they cannot be changed or undefined.
 - A valid constant name starts with a letter or underscore (no \$ sign before the constant name).
 - **Note:** Unlike variables, constants are **automatically global** across the entire script.

```
<?php
    define('GREETING', 'Welcome to IPLEIRIA!');
    echo GREETING;
?>
```



BIBLIOGRAPHY